Greedy Strategy-Based Swarm-Robot Dynamic Target Pursuit with a Lock Mechanism

Xihang Xiao
Department of Computer Science and Engineering
Southern University of Science and Technology
Shenzhen, China
12210923@mail.sustech.edu.cn

Zhitao Xu
Department of Computer Science and Engineering
Southern University of Science and Technology
Shenzhen, China
12211818@mail.sustech.edu.cn

Sirui Wu
Department of Computer Science and Engineering
Southern University of Science and Technology
Shenzhen, China
12210122@mail.sustech.edu.cn

***Abstract*—The dynamic target pursuit is a type of predator-prey problem, which is a significant area in swarm intelligence research that focuses on coordination and cooperation among multiple agents. Current methods like Particle Swarm Optimization-based Cooperative Coevolutionary Algorithm for Robots (CCPSO-R) use fitness functions to control predators, but lack direct guidance for predators from prey-related information, easily causing livelock. To address this, we propose a novel greedy strategy-based swarm-robot dynamic target pursuit algorithm with a unique lock mechanism, providing more direct guidance between prey and predators. Experiments demonstrate that it effectively resolves livelock, significantly improving capture success rates and reducing pursuit steps compared to CCPSO-R. Furthermore, to handle multi-target scenarios that hold greater application potential, we integrate two multi-task allocation strategies into the algorithm. This integration prevents deadlock issues arising from low predator-prey ratios and demonstrates strong adaptability across various scenarios. Overall, the proposed approach offers an efficient solution for both single and multi-target pursuit problems in dynamic environments.**

***Index Terms*—pursuit domain, lock mechanism, swarm intelligence, optimization, clustering**

I. Introduction

The pursuit domain problem, a classic predator-prey scenario, serves as a fundamental testbed for studying coordination techniques in swarm intelligence, as it was proposed by Benda et al. [1]. It involves predators executing strategic coordination to capture prey, offering a simple yet challenging framework for exploring emergent swarm behaviors. This problem is intrinsically linked to swarm optimization, as it requires the development of algorithms that enable efficient coordination and decision-making among multiple agents. The pursuit domain problem has been extensively studied due to its relevance in various applications, such as surveillance, search and rescue, and multi-robot systems.

Typically, pursuit domain problems include scenarios where predators either occupy the prey's position or surround it using four predators to block all orthogonal escape routes. Additionally, various assumptions about the environment are common, such as whether the world is bounded or infinite.

In this paper, we focus on the dynamic target pursuit by swarm robots in bounded diagonal grid worlds, involving scalable predator robot swarms and diverse prey types. Existing methods, such as the Particle Swarm Optimization-based Cooperative Coevolutionary Algorithm for Robots (CCPSO-R) [2], have shown effectiveness in solving this type of pursuit problem. However, CCPSO-R relies solely on the fitness function to guide predators, which can sometimes lead to livelock – a situation where predators move continuously but fail to capture the prey. To address this, we propose a unique lock mechanism integrated into a greedy algorithm. This mechanism provides more direct guidance for predators compared to CCPSO-R, thereby better addressing the livelock issue and demonstrating significant performance improvements. Further, we explore that under the lock mechanism, corner-based encirclement cases become common, thus, we incorporate an edge-related factor (f_{edge}) into the original fitness function to better leverage corner-based encirclement, though the benefits prove marginal. Additionally, we addressed multi-target scenarios by introducing two multi-task allocation algorithms to prevent deadlocks caused by low predator-to-prey ratios. Experiments revealed that combining these algorithms effectively resolves deadlocks and improves performance under specific conditions, although each algorithm has its own advantages in speed and adaptability.

The rest of the paper is organized as follows. First, the essential definitions, including the pursuit domain, prey logic, and detailed modeling of the baseline and proposed algorithms, are outlined in Section II. Then, the experimental methodologies, results, and in-depth discussions are presented in Section III. Finally, a summary of the findings and directions for future research are given in Section IV.

II. Preliminary & Methodology

A. Pursuit domain problem

The pursuit domain problem is a classic scenario in multi-agent systems where predators attempt to capture prey through strategic coordination. In this study, we focus on a pursuit domain within bounded diagonal grid worlds, involving scalable predator robot swarms and various prey types. The capture is defined as all available orthogonal neighbors of the prey robot being occupied either by predator robots or the grid boundary. This may differ from some existing work, like RL algorithms, where capture is defined as a predator occupying the prey's position. Key aspects of the pursuit game include: prey moving first, followed by predators in a fixed order to simplify collision avoidance; all agents having visible positions but no explicit communication; and implicit coordination among predators guided solely by a fitness function.

In [2], the fitness function for each robot considers the following metrics:

- **CLOSURE** $f_{\textbf{closure}}$: ensures the prey is enclosed within the convex hull of predator positions.
- **SWARM EXPANSE** $f_{\textbf{expanse}}$: encourages predators to concentrate around the prey.
- **UNIFORMITY** $f_{\textbf{uniformity}}$: promotes an even distribution of predators around the prey.
- **COLLISION AVOIDANCE** $f_{\textbf{repel}}$: prevents collisions among predators by maintaining safe distances.

The overall fitness function intergrates all these metrics in the following way:

$$f = f_{\text{repel}} \cdot (f_{\text{closure}} + f_{\text{expanse}} + f_{\text{uniformity}}) \tag{1}$$

B. Prey logic

To comprehensively evaluate the performance of pursuit algorithms and test their generality, we employ five different prey settings. Initially preys are randomly disparsed in the grid world. The prey types and their behaviors are as follows:

- **STILL PREY**: This type of prey remains stationary throughout the experiment, serving as a baseline to assess the predators' ability to encircle a non-moving target.
- **RANDOM PREY**: This prey moves to a randomly selected new position each time step, following a uniform distribution. Its unpredictable movement tests the adaptability of the predators.
- **LINEAR PREY**: Initially, this prey chooses a direction with the fewest predators from eight possible directions and moves straight in that direction. If it reaches the map's edge, it recalculates its direction based on the same criterion. However, if its path is blocked by a predator, it stops and waits to be encircled. This tests the predators' ability to intercept a target maintaining a consistent direction.
- **SMART LINEAR PREY**: Similar to linear prey, but when its path is blocked by a predator, it moves to the nearest unoccupied neighbor with the smallest angle deviation from its current direction and continues moving in that direction if no obstacles are present. This type represents a more challenging target that can adjust its path to evade predators.
- **TURTLE LINEAR PREY**: Similar to linear prey, but when there at least 2 directions are blocked, it will immediately stop moving like a turtle. This type can easily cause predators deadlock when the total numbers of preys and predators are similar.

In the single target scenario, the difficulty of capturing these preys can be intuitively ranked as: "still prey $<$ random prey $<$ linear prey $<$ turtle linear prey $<$ smart linear prey". Nevertheless, when it comes to the multi-target scenario, the difficulty ranking may vary since when the total number of predators is less than 4 times the total number of preys, deadlock can be caused in several situations, and still prey and turtle linear prey may be the hardest types to capture then. The above discussions will be verified by the following experiments and discussions in Section III.

C. Predator logic

1) Baseline design: We selected CCPSO-R [2] as our baseline. It effectively combines real-virtual robot co-evolution with the PSO algorithm [3], a well-established and efficient swarm intelligence technique. As shown in Algorithm 1, the algorithm guides predators using a fitness function detailed in Section II-A.

It involves N_s predators, which are also the real robots. Each predator maintains a population of N_p individuals, where all other individuals in the population are virtual robots. The virtual robots also participate in fitness evaluation and update their positions via the PSO algorithm. In doing so, they explore the action space of their corresponding real predator, guiding it to move to some optimal direction, to better encircle the prey. Parameter T_v in Algorithm 1 denotes the maximum number of virtual robots allowed in the vicinity of the corresponding real predator robot, typically within a $5*5$ grid or larger, depending on the environment.

2) Greedy strategy with proposed lock mechanism: During baseline PSO algorithm testing, we identified two key issues: (1) insufficient effectiveness of the fitness function in guiding predators, leading to PSO livelock problems, and (2) predators clustering together after prey escape, making subsequent captures more difficult.

To address these issues, we propose a Lock mechanism, as illustrated in Algorithm 2: when a predator comes within the 8-neighborhood of the prey, it stops following the fitness function and instead persistently tracks the prey's movements. This ensures that several predators maintain close proximity to the prey, ultimately facilitating corner-based capture.

3) Greedy strategy with f_{edge}: In the previous greedy algorithm experiment, we observed that since the predator and prey have identical movement speeds (each moving exactly one grid cell per turn), predators often rely on corner positions to capture prey.

To improve this algorithm design, we introduced a corner influence factor into the predator's movement guidance fitness

Algorithm 1 CCPSO-R Algorithm

```
Initialize real and virtual robot positions in an array p_robots shaped (N_s, N_p)
procedure CCPSO-R(p_robots, preys, grid, fitness_func)
    while prey not captured and time limit not reached do
        for each subpopulation do
            Re-evaluate the subpopulation due to environmental changes
            for each virtual robot do
                Update its position using rules based on PSO
                Evaluate the fineness together with the rest real predator robots
            end for
            if unique virtual robots < T_v then
                Re-initiate and re-evaluate virtual robots
            end if
            Update real predator robot's position based on virtual robots' best positions
            Evaluate the fitness of the real predator robot together with the rest real predator robots
            if the real predator robot becomes the global best then
                Re-initiate and re-evaluate the virtual robots
            else if the predator robot gets trapped in a deadlock then
                Add a random noise to the real predator robot position
                Re-evaluate the whole population
            end if
        end for
        if the real predators swarm get trapped in a local optimum then
            Add random noises to all the real predator robots' positions
            Re-evaluate the whole population
        end if
    end while
end procedure
```

Algorithm 2 Naive Predator Movement Algorithm

```
procedure NAIVE(p_robots, preys, grid)
    Ns ← shape(p_robots)[0]                    ▷ Robot count
    SN ← {eightdirections}                     ▷ 8-neighborhood
    function COLLIDE(pos)
        if ∃ robot at pos OR ∃ prey at pos then
            return True
        end if
        return False
    end function
    function LEGALSTEPS(pos)
        steps ← {[0, 0]}                       ▷ Include stationary option
        for each orthogonal step step ∈ SN[0 :: 2] do
            target ← pos + step
            if target within grid bounds AND not COLLIDE(target) then
                steps.append(step)
            end if
        end for
        for each diagonal step step ∈ SN[1 :: 2] do
            target ← pos + step
            hori ← [step[0], 0], vert ← [0, step[1]]
            if target within grid bounds AND not COLLIDE(target AND)
                not (COLLIDE(pos+hori) AND COLLIDE(pos + vert)) then
                steps.append(step)
            end if
        end for
        return steps
    end function
    for each predator i ∈ 1 to Ns do
        steps ← LEGALSTEPS(p_robots[i, 0])
        targets ← p_robots[i, 0] + steps
        Calculate distances from targets to preys
        step ← steps[argmin(min(distances, axis = 1))]
        p_robots[i, 0] ← p_robots[i, 0] + step
    end for
end procedure
```

function. This enhancement aims to better direct predators around corners to surround prey and improve performance. Since the corner capture scenario ultimately positions the prey in a corner with predators occupying the two adjacent points, our objective is to guide predators toward these two points (denoted as A and B) based on the prey's position.

Let dc_i represent the distance from the i-th prey to its nearest corner, dd denote the length of the map's diagonal, and dp_{ij} indicate the distance from the j-th predator in the i-th subgroup to the nearer of points A or B. The corner influence factor is calculated as:

$$f_{\text{edge}} = e^{(dc_i/dd)} + e^{(dp_{ij}/dd)} \tag{2}$$

The integrated fitness function then becomes:

$$f = f_{\text{repel}} \cdot (f_{\text{closure}} + f_{\text{expanse}} + f_{\text{uniform}} + f_{\text{edge}}) \tag{3}$$

4) Incorporate clustering strategies for multi-target pursuit: In previous optimizations involving the addition of f_{edge}, we observed limited improvement in performance. During testing of multi-prey pursuit scenarios, we identified a critical efficiency issue in the algorithm: the current implementation lacks task allocation logic among predators. All predators simply pursue the nearest prey, which can lead to multiple predators targeting the same prey, resulting in inefficiency. To address this, we designed a clustering method to rationally allocate predator resources and improve efficiency.

Density-Based Adaptive Allocation: Let N_s represent the number of predators, and let n_{prey} represent the number of preys. First, we determine the minimum number of predators required for allocation based on the floor value of N_s/n_{prey}. If this value is less than 2, we initially group predators in pairs to capture some prey and then recalculate the allocation. Next, we measure the number of predators within a certain range (20% of the side length, chosen based on actual animal perception capabilities in small ecosystems where typical sensing radii range from 10% to 30% of the map size) around each prey, denoted as predators_around, and calculate f_{density}:

$$f_{\text{density}} = 1 - e^{predators_around} \tag{4}$$

The number of predators required for each prey is then calculated by:

$$\text{required_robots} = \left\lfloor \frac{N_s}{n_{\text{prey}}} \right\rfloor + f_{\text{density}} \tag{5}$$

Prey with higher f_{density} values are more likely to be assigned the remaining predators. The range of f_{density} is between 0 and 1, ensuring that the minimum number of predators needed for allocation ($\lfloor N_s/n_{\text{prey}} \rfloor$) is not affected, while allowing prioritization of prey that should receive additional predators based on their density.

For example, in a scenario with 8 predators and 3 prey (A, B, C), where prey A has 4 nearby predators, B has 3, and C has 2, the algorithm would first allocate 2 predators to each prey (the floor value of 8/3). The remaining 2 predators would then be preferentially assigned to prey A and B.

K-Means Clustering Allocation: In practice, when the preys adopt static or turtle linear strategy, it is impossible for predators that adopt greedy or density-based adaptive allocation strategy to capture the preys. So, we come up with the k-means cluster strategy. Let N_s represent the number of predators, and n_{prey} represent the number of prey. First, we cluster these predators into $\lfloor \frac{N_s}{k} \rfloor = c$ clusterings, where k in an integer and often chosen as 4. Then, each centroids of the clusterings are mapped to a prey with stable matching if $c \leq n_{prey}$, otherwise each prey are mapped with stable matching and the predators in the clustering whose centroids are leftout will be mapped to the closest prey. Third, the predators in the clusterings whose centoids are mapped to a prey will be mapped to that prey. Finally, each predator will try to capture the prey they are mapped to individually.

Algorithm 3 Density-Based Adaptive Allocation Algorithm

Require: p_robots: predator positions (shape $Ns \times Np \times 2$)
Require: $preys$: prey positions (shape $N_prey \times 2$)
Require: $grid$: grid size
Ensure: Updated p_robots positions

```
real_robots ← p_robots[:, 0]
Compute dist_matrix between all robots and preys
assignments ← zeros(Ns), prey_status ← zeros(N_prey)
robots_per_prey ← ⌊auto_cluster()⌋
if Ns < 2N_prey and robots_per_prey == 1 then
    Group robots into pairs/triples
    for each group do
        Assign to nearest uncaught prey
        if group captured prey then
            Mark prey as caught
        end if
    end for
else
    Assign robots_per_prey robots to each prey
    for each unassigned robot do
        Assign to closest prey
    end for
end if
for each robot i do
    if assigned prey is caught then
        Reassign to nearest uncaught prey
    end if
    Compute legal steps
    Move toward assigned prey
end for
return p_robots
```

III. Experiments & Results

A. Baseline experiment(CCPSO-R)

The results of the baseline experiment are shown in Fig. 1. Normally, we consider over 100 moves in 30*30 grid and over 500 moves in 150*150 grid as failure cases. Generally, CCPSO-R performs well within 250 moves, even for the larger 150*150 grid. However, signs of instability are observed in tests on the 30*30 grid with 8 predators, where in some trials, predators fail to capture the prey even within 300 moves. Analysis reveals that these failures are attributed to livelock phenomena.

This differs from deadlock, where robots halt completely and can be resolved by adding random noise, as indicated in Algorithm 1. Livelock occurs when predators, after an initial attempt to encircle the prey, form a dense cluster and fail to disperse effectively. The predators continue moving toward the prey's last known position but struggle to reposition for a successful capture. The primary reason is that the fitness function's guidance to predators is delayed due to the indirect path through virtual robots, and the identical movement speed of predators and the prey further impacts real-time pursuit performance. To address these issues, we propose enhancements to the fitness function and introduce the lock mechanism, as described in Section II-C2 and II-C3, to prevent premature escape and improve capture efficiency.

Algorithm 4 K-Means Clustering Allocation

Require: p_robots: predator positions (shape $N_s \times 2$)
Require: $preys$: prey positions (shape $n_{\text{prey}} \times 2$)
Require: k: cluster size (e.g., $k = 4$)
Ensure: Assignment of each predator to a prey

```
c ← ⌊N_s/k⌋                                   ▷ number of clusters
Cluster p_robots into c clusters using K-means, obtain
cluster centroids
if c ≤ n_prey then
    Stable matching maps each centroid to a unique prey
    for each cluster do
        if cluster is assigned then
            Assign all robots in the cluster to the prey
        end if
    end for
else
    Stable matching maps each prey to a centroid
    for each cluster do
        if cluster is assigned then
            Assign all robots in the cluster to the prey
        end if
    end for
end if
for each robot i do
    if assigned prey is caught then
        Reassign to nearest uncaught prey
    end if
    Compute legal steps
    Move toward assigned prey
end for
```

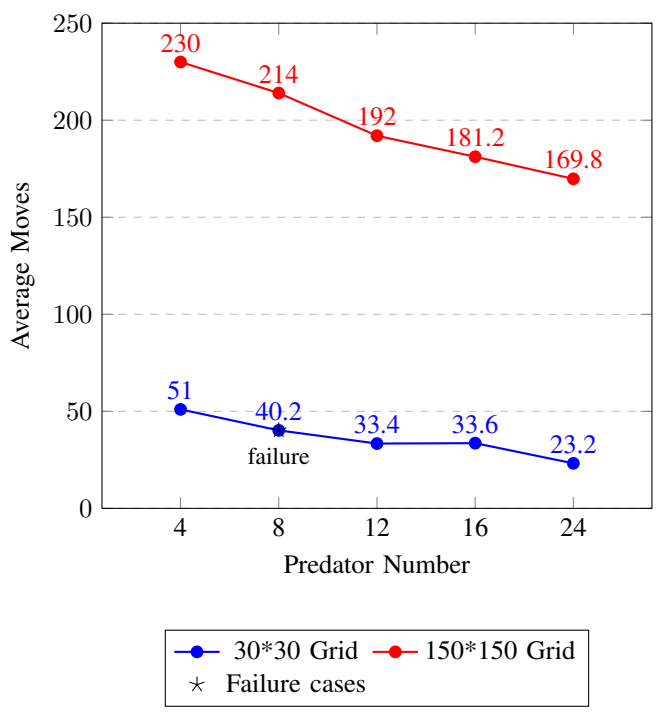

Fig. 1. Average moves to capture prey with different numbers of predators in different grid sizes. Failure cases found in individual tests on 30*30 grid with 8 predators.

B. Greedy experiment with lock mechanism

The results of greedy experiment with lock mechanism are shown in Fig. 2.

When the number of moves exceeds 100, the trial is considered a failure. As shown in Fig. 2, compared with the CCPSO-R baseline, the proposed algorithm demonstrates significant improvement. This enhancement primarily stems from the Lock mechanism, which prevents predators from blindly following the fitness function. Instead, individual predators autonomously select optimal positions for pursuit based on their current locations.

However, failure cases still exist as observed in Section II-C3. The algorithm retains optimization potential, particularly through the incorporation of f_{edge} into the fitness function.

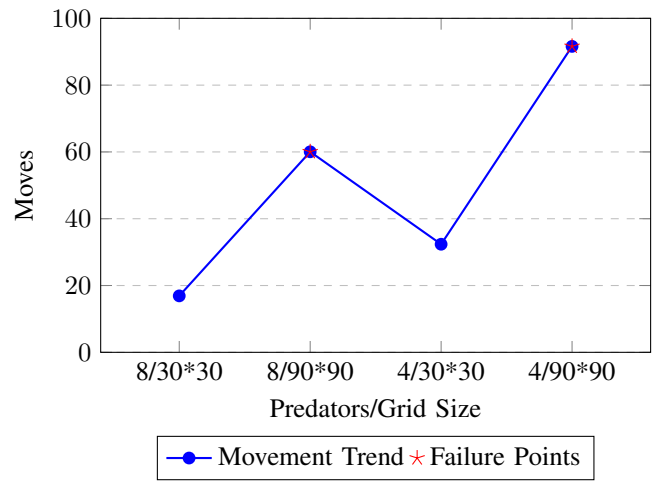

Fig. 2. Move count trends under different configurations. Red stars denote trials exceeding 100 moves (failures).

C. Greedy experiment with f_{edge}

The results and comparisons of greedy experiment with f_{edge} are shown in Fig. 3. We observe that the introduction of the f_{edge} component leads to higher capture efficiency, albeit with marginal improvement. The analysis reveals the following key factors:

- **Randomness in initialization**: Both prey and predators are randomly distributed across the entire grid, introducing significant stochasticity in the experimental outcomes.
- **Early predator locking**: In many cases, predators successfully lock onto prey outside the effective range of f_{edge} and maintain pursuit until capture occurs. The f_{edge} mainly concentrates more predators around boundary-positioned prey but provides limited benefit for actual capture (which typically requires only two or three predators).
- **Redundant guidance**: The original fitness function already possesses the capability to guide predators toward cornered prey. The f_{edge} merely enhances this boundary guidance effect without substantially improving capture performance.

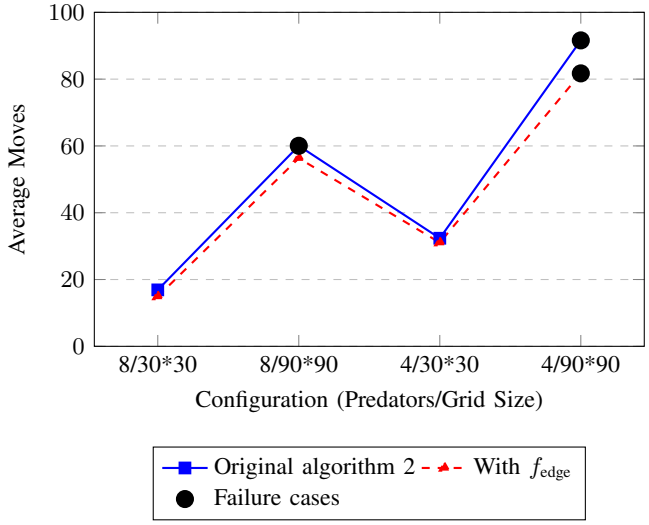

Fig. 3. Performance comparison between original algorithm 2 and f_{edge}-enhanced algorithm 2. Black dots indicate trials exceeding 100 moves (failures). The dashed red line shows consistent but marginal improvement across all configurations.

D. Greedy experiment with clustering

The comparative results of the greedy experiment with clustering are illustrated in Fig. 4. With the incorporation of the allocation strategy, our algorithm effectively resolves the uneven predator resource distribution problem, demonstrating significant improvements in capture efficiency. The enhancement is most pronounced when the predator-to-prey ratio falls between 2:1 and 3:1. The underlying reasons can be analyzed as follows:

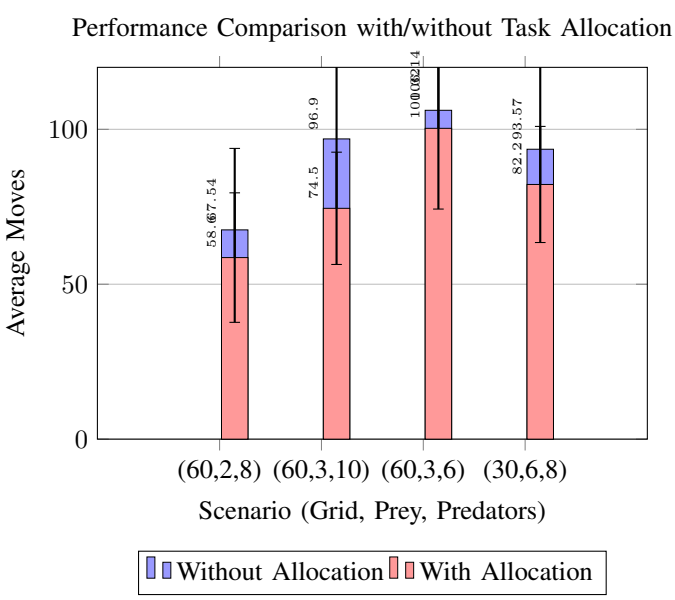

Fig. 4. Performance comparison with widened columns. Each column shows the average moves with standard deviation error bars. The task allocation approach (red) consistently outperforms the original algorithm 2 (blue) across all scenarios.

- **High ratio scenarios (4:1 or higher)**: The abundance of predators enables comprehensive prey capture even without explicit allocation, resulting in marginal performance differences between strategies.
- **Low ratio scenarios (below 2:1)**: Both the new allocation strategy and original approach prioritize partial prey capture while ignoring others, leading to comparable final efficiency.
- **Optimal ratio range (2:1 to 3:1)**: The moderate predator count allows the new strategy to maintain simultaneous pursuit of all prey, whereas the original strategy leaves some prey unassigned. This necessitates additional movement after initial captures, making the allocation strategy particularly effective in this regime.

The current strategy exhibits limited adaptability, as it lacks dynamic adjustment mechanisms based on environmental factors E (e.g., prey movement patterns). We hypothesize that an adaptive formulation

$$\alpha(t) = \alpha_0 \cdot e^{-kt} \cdot E(t) \tag{6}$$

could further improve performance, where $E(t)$ represents environmental variables and k is the adaptation rate. This remains an open research question for future investigation.

E. Clustering experiments with different clustering strategies

The difference between any clustering strategy and the greedy strategy is similar. But there are differences between each clustering strategy. The comparative results of clustering experiments with different strategies are shown from Fig. 5 to Fig. 8. The density-based adaptive allocation strategy gains a better performance than the greedy strategy with the increasing ratio between the number of predators and the number of preys (predator-prey ratio), while they have a similar performance when the ratio is small. The underlying reasons can be analyzed as follows:

- **Small predator-prey ratio** (≤ 1): The cluster kmeans strategy can perform a similar process to the density-based adaptive allocation strategy for there are a small number of clusters.
- **Large predator-prey ratio** (> 1): Firstly, the cluster kmeans strategy tends to cluster most of predators into one cluster, leaving few predators (with number ≤ 2) to catch the rest preys, which always tends to fail. So, the cluster kmeans strategy can be trusted as the strategy captures only 1 prey each round. It is obvious that it can have lower efficiency than the density-based adaptive allocation strategy, which captures different preys in parallel. Furthermore, the cluster kmeans strategy may cluster the predators on the edge between 2 clusters into one of the 2 clusters but different from the previous cluster. This can cause the predator to vibrate between 2 clusters but not get closer to any of the centroids of the clusters. Thus, such predators will never get close to the preys and capture them.

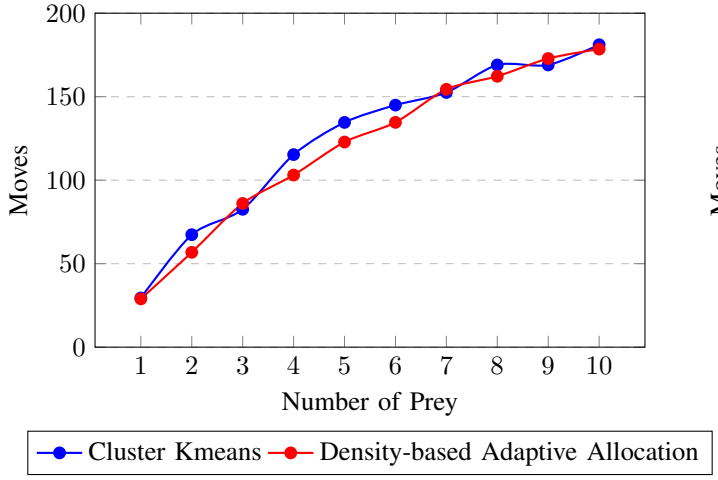

Fig. 5. Average Moves vs Number of Preys, with Number of Predators = 4.

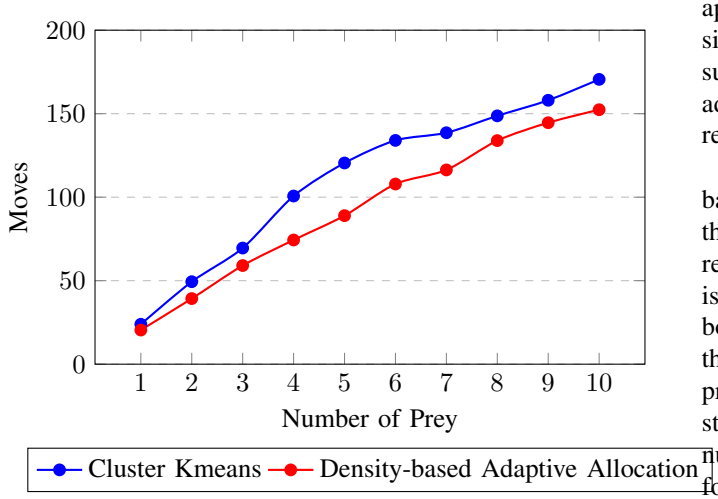

Fig. 6. Average Moves vs Number of Prey, with Number of Predators = 6.

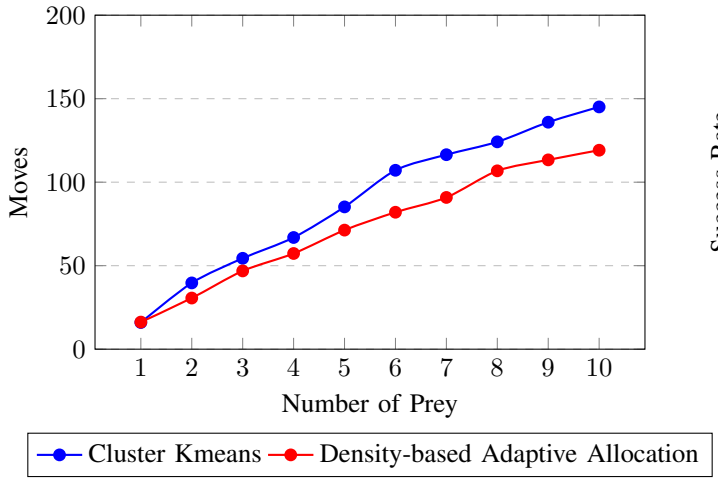

Fig. 7. Average Moves vs Number of Prey, with Number of Predators = 8.

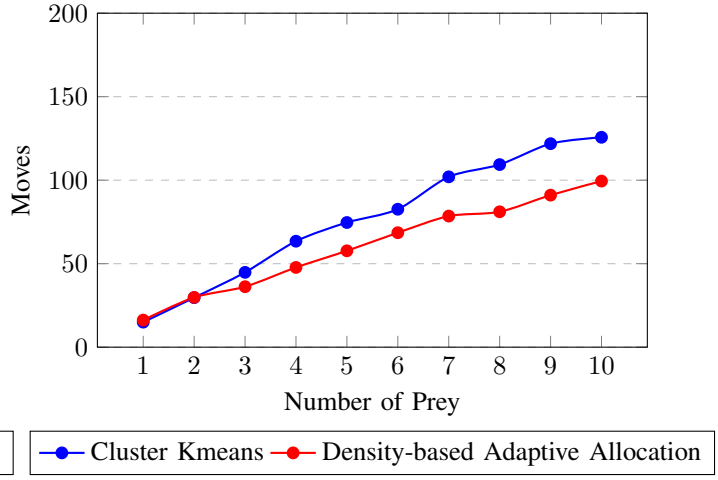

Fig. 8. Average Moves vs Number of Prey, with Number of Predators = 10.

Above we consider the most general situations, where preys apply the smart linear strategy. When it comes to special situations of static or turtle preys, the difference between the success rates of kmeans cluster strategy and density-based adaptive allocation strategy is very huge. The comparative results are shown in Fig. 9 and Fig. 10.

The underlying reason can be analyzed that the density-based adaptive allocation strategy tends to push the preys into the corner so they can easily be captured. But the preys will remain unmoved when the capture begins or when the situation is dangerous. So, with no possibility of making use of the boundary to capture the prey, the predators can only capture the prey by surrounding it from the 4 directions. But the predators that adopt cluster density-based adaptive allocation strategy will try to capture the nearest prey. So, when the number of prey increases, there are not enough predators for each prey to be captured. So, the density-based adaptive allocation strategy will fail. But in cluster kmeans strategy, a sufficient number of predators will try to capture specific prey. Thus, it can succeed easily.

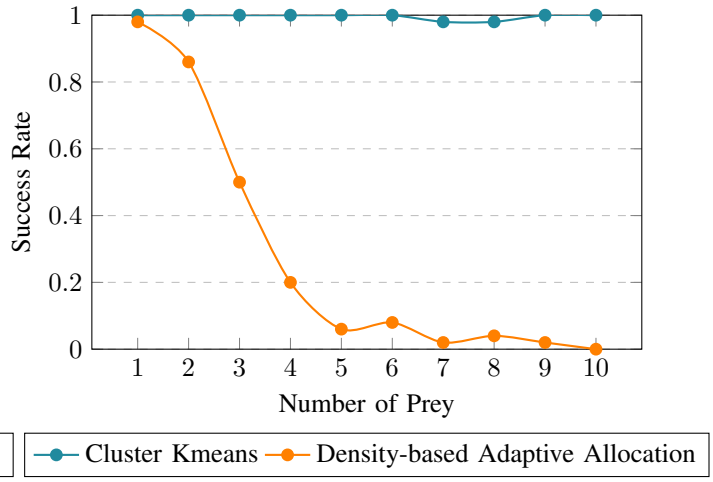

Fig. 9. Success Rate of Prey vs Number of Prey with static prey.

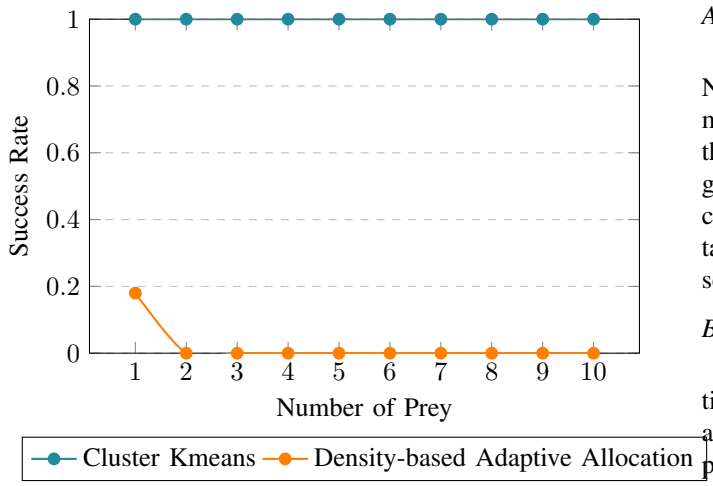

Fig. 10. Success Rate of Prey vs Number of Prey with turtle linear prey.

IV. Conclusions & Future works

In this paper, we have focused on the dynamic target pursuit by swarm robots. Our proposed greedy-based algorithm with a unique lock mechanism has shown significant improvements over the CCPSO-R baseline. It effectively resolves the livelock issue and enhances capture success rates with fewer moves and greater stability. Moreover, we observed that corner-based captures frequently occur due to the identical speeds of predators and preys. To leverage this observation and enhance performance, we added an edge-related factor (f_{edge}) to the overall fitness function. However, this attempt yielded only minimal improvements.

In multi-prey scenarios, we identified deadlock issues arising from insufficient predator numbers, particularly with still and turtle linear preys. To tackle this, we introduced two multi-task allocation strategies: density-based adaptive allocation and K-means clustering allocation. The former demonstrated marked improvements in capture success rates when the predator-to-prey ratio was between 2:1 and 3:1, but lacked adaptability beyond this range. The latter showed higher adaptability with consistent success rates across various ratios, but its computational efficiency is lower due to its sequential capture approach. Besides, K-means clustering exhibited a tendency to frequently reassign predators near cluster boundaries, causing predators to oscillate between two prey targets and hindering effective participation in captures. Moving forward, we will explore the integration of both strategies to leverage their respective advantages.

This study on lock mechanisms and multi-task allocation strategies serves as an initial step. Building on the achievements of the greedy algorithm, we plan to explore the application of other swarm intelligence algorithms, such as Brain Storm Optimization (BSO) [4], to further enhance the algorithm's performance. Besides, we believe that other directions like deep learning, reinforcement learning, and so on, hold great potential for enhancing the pursuit performance.

A. Deep Learning Direction

We plan to introduce Spatio-Temporal Graph Convolutional Networks (ST-GCN) [5] to dynamically analyze prey movement trajectories and swarm spatial relationships, replacing the static density evaluation function (f_{density}). This model will generate adaptive encirclement strategies in real-time, significantly enhancing responsiveness to complex prey behaviors, targeting a 15% improvement in capture rates in dynamic scenarios.

B. Reinforcement Learning Direction

We plan to employ Multi-Agent Proximal Policy Optimization (MAPPO) [6] to jointly optimize long-term coordination among predators. By designing state spaces (prey/predator positions) and reward functions (distance reduction + encirclement rewards), end-to-end training of swarm collaboration strategies will fundamentally resolve deadlocks caused by turtle-like prey, aiming to reduce average move counts by 20%.

Additionally, we aim to improve the algorithm's scalability and adaptability, enabling it to handle more complex scenarios such as dynamic target encirclement in environments with obstacles.

References

[1] M. BENDA, "On optimal cooperation of knowledge sources : An empirical investigation," *Technical Report, Boeing Advanced Technology Center*, 1986. [Online]. Available: https://cir.nii.ac.jp/crid/1570572700270838272

[2] L. Sun, C. Lyu, and Y. Shi, "Cooperative coevolution of real predator robots and virtual robots in the pursuit domain," *Applied Soft Computing*, vol. 89, p. 106098, 2020. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S1568494620300387

[3] Y. Shi and R. Eberhart, "A modified particle swarm optimizer," in *1998 IEEE International Conference on Evolutionary Computation Proceedings. IEEE World Congress on Computational Intelligence (Cat. No.98TH8360)*, 1998, pp. 69–73.

[4] Y. Shi, "Brain storm optimization algorithm," in *Advances in Swarm Intelligence*, Y. Tan, Y. Shi, Y. Chai, and G. Wang, Eds. Berlin, Heidelberg: Springer Berlin Heidelberg, 2011, pp. 303–309.

[5] S. Yan, Y. Xiong, and D. Lin, "Spatial temporal graph convolutional networks for skeleton-based action recognition," 2018. [Online]. Available: https://arxiv.org/abs/1801.07455

[6] C. Yu, A. Velu, E. Vinitsky, J. Gao, Y. Wang, A. Bayen, and Y. Wu, "The surprising effectiveness of ppo in cooperative, multi-agent games," 2022. [Online]. Available: https://arxiv.org/abs/2103.01955